

OCR GCE A-LEVEL H446 — COMPUTER SCIENCE

NON-EXAMINED ASSESSMENT (NEA)

Air Traffic Control Management Simulator

Candidate Name: Haayed Aslam

Candidate Number:

School: Wilson's School

Centre Number:

Mark Achieved: 68 / 70 (A*)

Year: 2024–25

A Python simulation of a 2D Air Traffic Control landing system,
built with `tkinter` and featuring real-time collision priority queuing.

Contents

1	Analysis	3
1.1	Project Definition	3
1.2	Stakeholders	3
1.3	Research	4
1.3.1	Research Methodology	4
1.3.2	Existing Solutions	5
1.4	Features of the Proposed Solution	7
1.5	Limitations of the Proposed Solution	8
1.6	Requirements & Success Criteria	8
2	Design	10
2.1	Systems Diagram	10
2.2	Class Diagrams	11
2.3	User Interface Design	11
2.3.1	Start-up screen	12
2.3.2	Radar interface (idea 1)	13
2.3.3	Radar interface (idea 2)	14
2.4	Algorithms	15
2.4.1	Render holding patterns	15
2.4.2	Render planes	16
2.4.3	Spawn new plane	17
2.4.4	Check for new emergency	18
2.4.5	Recalculate plane coordinates	19
3	Development	20
3.1	Overview	20
3.2	Iteration 1 — Start-up Screen	20
3.2.1	Scope	20
3.2.2	Key Implementation Details	20
3.2.3	Iteration 1 — Review	21
3.3	Iteration 2 — Radar Interface	22
3.3.1	Scope	22
3.3.2	Key Implementation Details	22

3.3.3	Debugging & Error Handling	24
3.3.4	Iteration 2 — Review	24
3.4	Iteration 3 — Core Simulation Logic	25
3.4.1	Scope	25
3.4.2	Plane Class	25
3.4.3	Message Class	25
3.4.4	Debugging & Error Handling	27
3.4.5	Iteration 3 — Review	27
3.5	Iteration 4 — Advanced Features	28
3.5.1	Scope	28
3.5.2	Priority Queue Algorithm	28
3.5.3	Debugging & Error Handling	29
3.5.4	Iteration 4 — Review	29
4	Testing	31
4.1	Acceptance Testing (Post-Development)	31
5	Evaluation	32
5.1	Usability Features	32
5.1.1	Start-up Screen	32
5.1.2	Main Radar Screen	32
5.1.3	Aircraft Recognition	32
5.2	How Well Does the Solution Match the Success Criteria?	32
5.2.1	Requirements Fully Met (8/12)	33
5.2.2	Requirements Partially Met (3/12)	33
5.2.3	Requirements Not Met (1/12)	33
5.3	Maintenance	33
5.4	Limitations and Future Development	34

Chapter 1

Analysis

1.1 Project Definition

Air Traffic Control (ATC) is notorious for being one of the most stressful jobs in the world, requiring strong communication and workload management skills under high-pressure situations, with potentially thousands of lives on the line if mistakes are made. Large commercial airports such as London Gatwick (LGW) operate hundreds of flights every day, making safety and real-time decision-making even more crucial. This is what my ATC training simulator aims to address.

The system will be a classroom-based simulation tool used in ATC schools. Instructors can simulate various landing scenarios on this system, such as what to do if more aircraft are ready to land than the maximum landing rate for that runway. The aim is that, by demonstrating these scenarios visually, the confidence and understanding of the students will improve, leading to fewer mistakes on the job and hence improving overall safety within aviation.

The project is tailored specifically to ATC training for a single runway civilian airfield such as LGW, as detailed below. There is the potential for the program to also be developed into a commercial-based automated system for use in real-life ATC towers. I will investigate the suitability of this idea during my discussions with stakeholders

1.2 Stakeholders

- **National Air Traffic Services (NATS):** the main provider of ATC training in the UK, so a potential buyer for use as an off-the-job training tool.
- **UK Civil Aviation Authority (CAA):** the regulatory body for UK air and space operations, which would need to test and verify compliance with real-life protocols (e.g. Rights of Way) before authorising the simulator.
- **LGW and other single-runway airports:** airport authorities who may use the

system to re-train existing ATC staff.

- **ATC training instructors:** they would use the simulator to deliver higher-quality instruction via tutorials and live demonstrations of different scenarios.
- **ATC training students:** the primary end-users, who interact with the system by experimenting with different scenarios, improving their confidence to make decision under pressure in the ATC tower.
- **Airline pilots and flight school students:** Flight Training Organisations (FTOs) such as Skyborne may use the system as part of ATPL syllabus modules covering ATC radio communication and holding-pattern procedures.

1.3 Research

1.3.1 Research Methodology

ChatGPT was used to generate an initial list of existing ATC simulators (prompt: "Please give a list of Air Traffic Control simulators"), from which two were shortlisted for analysis. The MoSCoW prioritisation framework (Fig. 1.2) was then used to categorise all desired features. Stakeholder surveys ran in parallel to capture user requirements.



Figure 1.1: The MoSCoW framework for prioritising features and user requirements

1.3.2 Existing Solutions

Solution 1: ATC-SIM

ATC-SIM is a basic 2D web-based ATC simulator. Key features:



Figure 1.2: ATC-SIM's radar-style user interface when first loaded. Each aircraft is a small white square showing flight ID and heading

BAW397 27L 83=	A320 To: EPM	
VIR347 27L 83=	B789 To: WOD	
SAS886 27L 83=	B738 To: EPM	
VIR679 27L 83=	B789 To: EPM	
DAL010 27L 83=	B763 To: BIG	
BEE2133 27L 83=	DH8D To: BIG	
DAL032 27L 83=	B763 To: EPM	
AAL195 27L 83=	B772 To: OCK	
BAW203 27L 83=	B788 To: WOD	
BAW709 294° 6000=	A319 Arrival	
BAW584 250° 4000=	A321 Arrival	
BAW292 27L 83=	B744 To: BPK	
VIR181 27L 83=	B789 To: BPK	
ACA881 27L 83=	B77W To: OCK	
SIA373 27L 83=	B77W To: BIG	

Figure 1.3: Flight strips displaying FlightID, aircraft type, altitude, and current heading

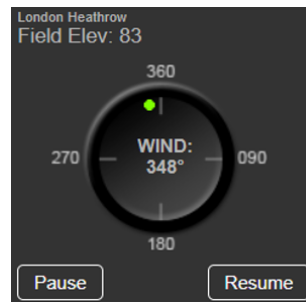


Figure 1.4: ATC-SIM's instruments panel

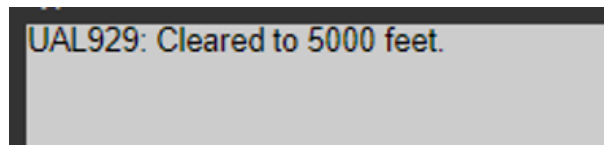


Figure 1.5: Example command in message box - clearing a departing aircraft for take-off

Solution 2: IVAO Aurora

IVAO Aurora is an advanced ATC client used by virtual aviation networks. Key features:



Figure 1.6: Aurora's user interface when first loaded (UK airspace). More sophisticated radar with sector boundaries and flight plan data



Figure 1.7: Aurora's radar-style interface when EGLL airport is selected

Stakeholder Surveys

I held Google Forms surveys with 6 stakeholders. Most agreed that an ATC simulator is an excellent tool for training. Most were strongly against the idea of a fully-automated system that could replace the need for humans in a real control tower. Thus, I have decided that a commercial version of this system for real towers will not be suitable due to potential issues surrounding accountability if the system makes an error. Instead, the program will be restricted to just a simulator tool for use only in NATS-approved ATC training schools.

1.4 Features of the Proposed Solution

Feature	Description	Priority
Start-up screen	Configurable sliders for simulation speed, total planes, spawn rate, emergency probability, and weather conditions	Must
2D radar GUI	Radar-style interface showing one runway and all planes as dots	Must
Holding patterns	Two labelled circular holding patterns (Level 1 and Level 2) separated by 1000 ft	Must
Settings sliders	Ability for user to adjust emergency rates, aircraft spawn rates, and weather conditions	Must

Feature	Description	Priority
Communications box	Output box displaying ATC-to-cockpit messages (spawn, land, emergency, end)	Must
Program end	System auto-halts when all planes have landed	Must
Flight strips	Strips next to each dot showing flight ID, fuel remaining and wait time	Should
Sound effects	Alarm on new emergency detection; alarm on impending collision	Should
Runtime buttons	Pause, Speed Up, Slow Down	Should
Collision detection & avoidance	Warn user of impending collisions; redirect aircraft to safe slot	Should
Heading indicator	Compass in corner of GUI; optional wind direction arrow	Could
Plane removal	Landed planes removed from radar	Could

1.5 Limitations of the Proposed Solution

- System is restricted to 2D; no 3D radar interface.
- Only one runway configuration is supported.
- No take-off simulation is included.
- Aircraft teleport between holding patterns (no animated transition path).
- No use of approach aids (e.g. Instrument Landing System).

1.6 Requirements & Success Criteria

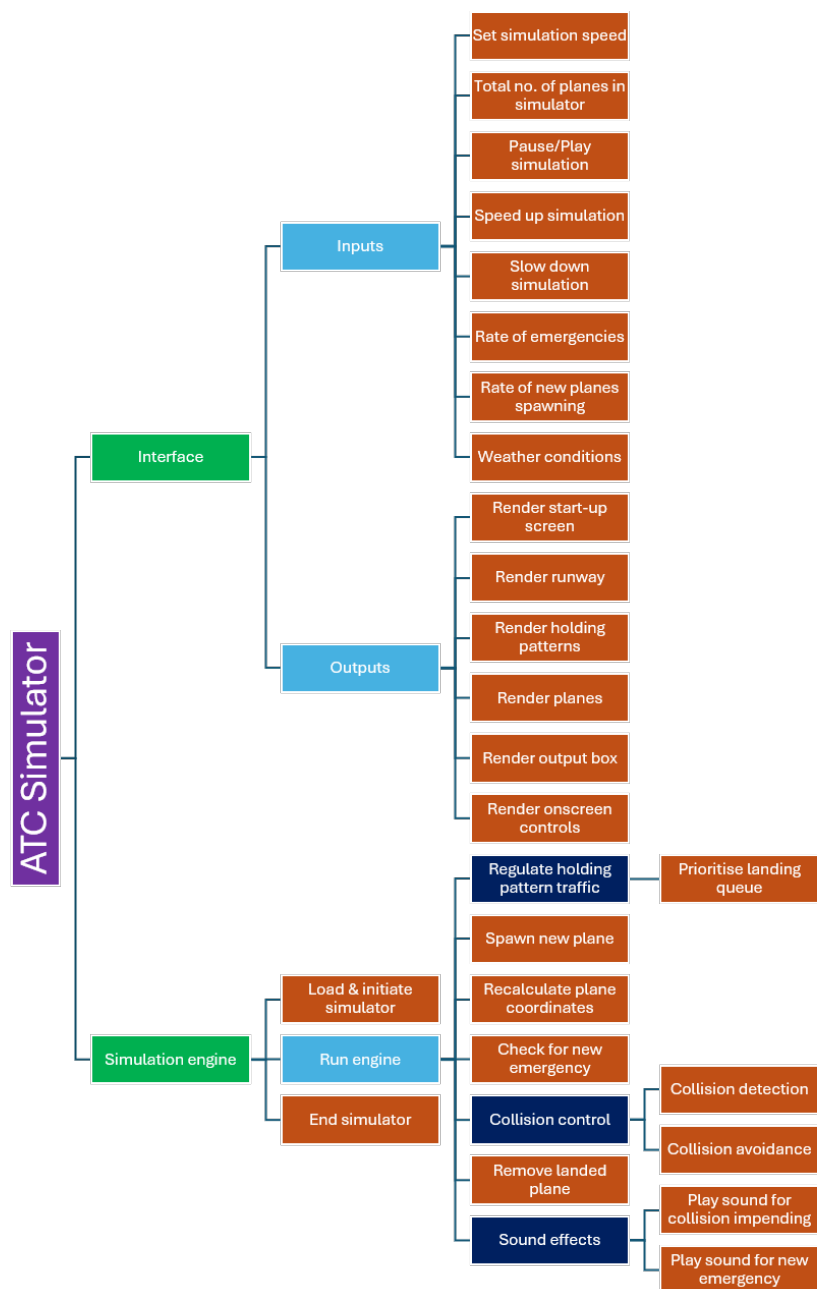
No.	Req.	Success Criteria	Justification
1	Start-up screen	Display screen with simulator name; sliders for speed, planes, spawn rate, emergencies, weather; Start button	Allows user to configure all parameters before commencing the simulation

No.	Req.	Success Criteria	Justification
2	2D radar interface	2D GUI showing one runway and all planes as dots	Allows user to observe all simulated events
3	Holding patterns	Two labelled circular holding patterns, vertically above each other, 1000 ft separation	Complies with Safe Separation Standards; mimics real holding circuits
4	Settings sliders	Mouse-operable sliders for emergencies, plane count, weather	Enables instructor to configure real-time scenarios
5	Output box	Messages for new plane, emergency, plane landed, end of simulation; future: collision warning	Mirrors real-life ATC text communications
6	Program end	System terminates when all planes have landed	Signals end of scenario; allows reset for new exercise
7	Flight strips	FlightID, fuel remaining, wait time displayed beside each dot	Allows user to track per-plane data and understand landing priority decisions
8	Sound effects	Alarm on emergency; alarm on impending collision	Trains students to respond to audio alerts as they would on the job
9	Runtime buttons	Pause, Speed Up, Slow Down	Enables instructor to freeze simulation for discussion or control pace
10	Collision detection	Warn via output box; redirect aircraft to safe slot	Trains students to manage collision scenarios
11	Heading indicator (compass)	Cardinal directions displayed; optional wind arrow	Provides spatial reference; wind arrow adds weather realism
12	Plane removal	Landed planes removed from radar	Reduces clutter once plane is no longer the controller's responsibility

Chapter 2

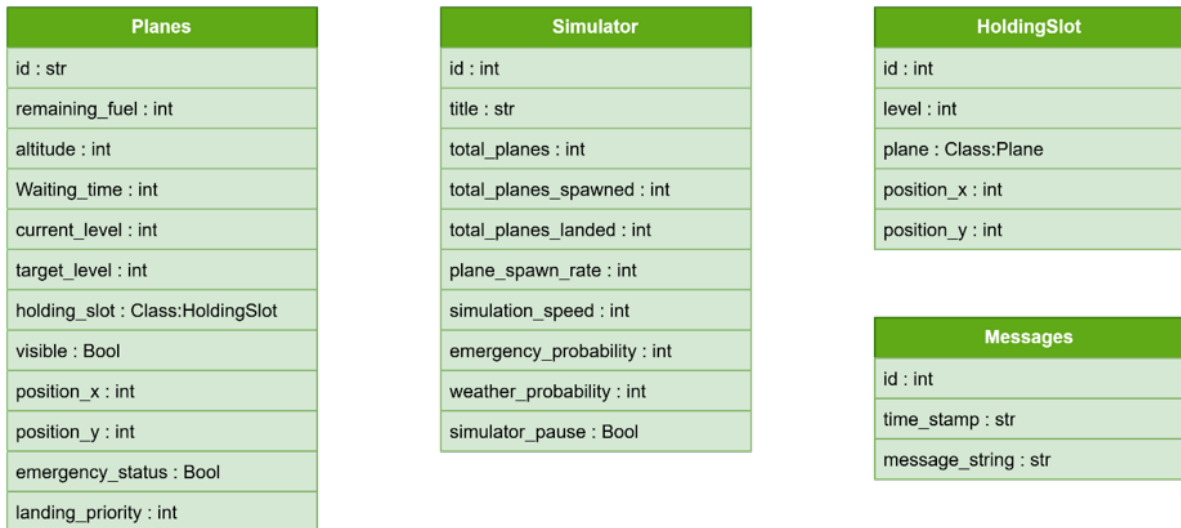
Design

2.1 Systems Diagram



2.2 Class Diagrams

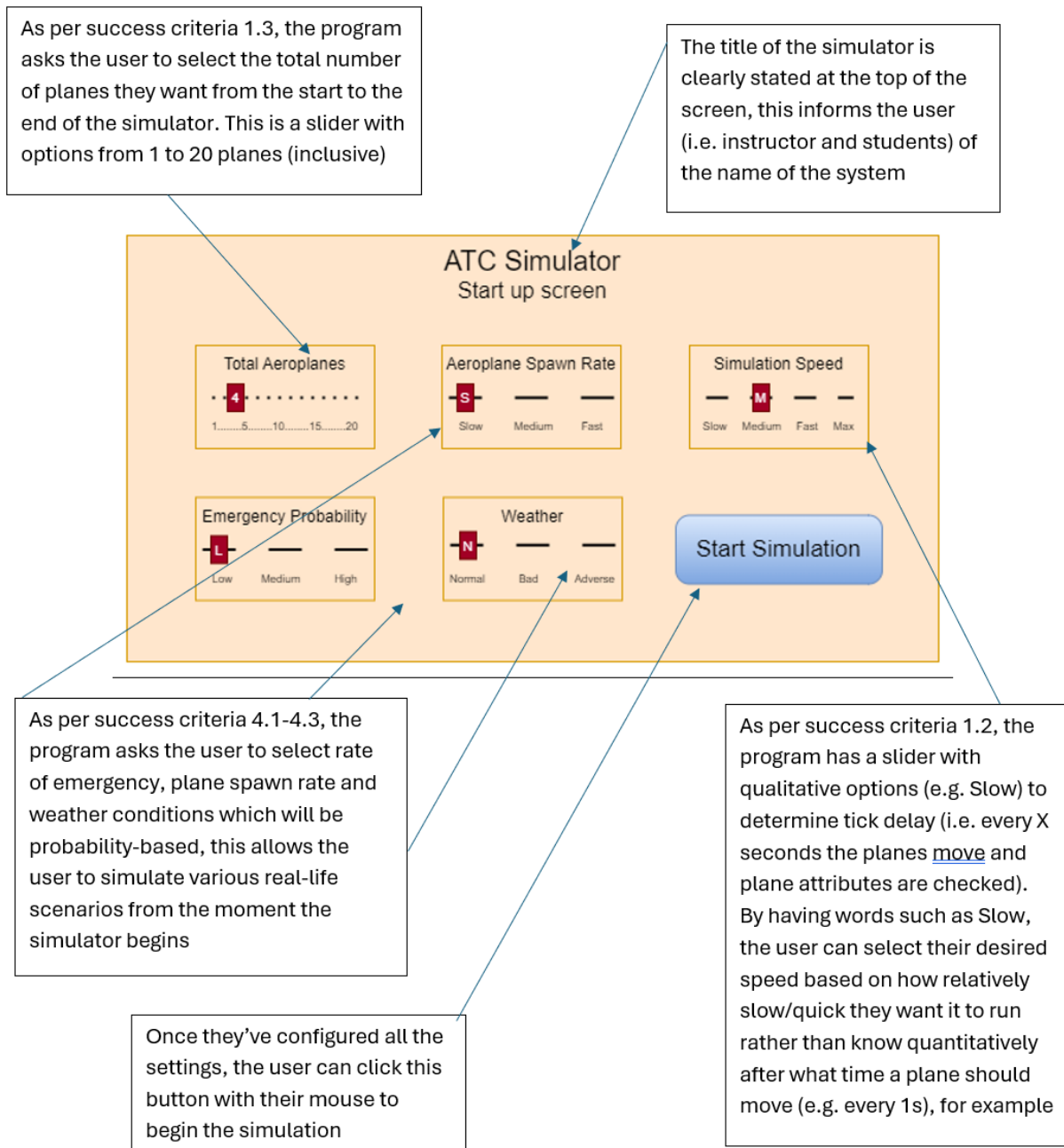
- **Planes** — stores all per-aircraft attributes (flightID, fuel, wait time, holding level, emergency status, position coordinates, colour, flight strip).
- **Simulator** — the master class managing the run engine, holding-slot allocation, landing-queue prioritisation, and all rendering functions.
- **HoldingSlot** — represents a position in Level 1 or Level 2, tracking occupancy.
- **Messages** — encapsulates all output-box message types (spawn, land, emergency, level change, end of simulation).



2.3 User Interface Design

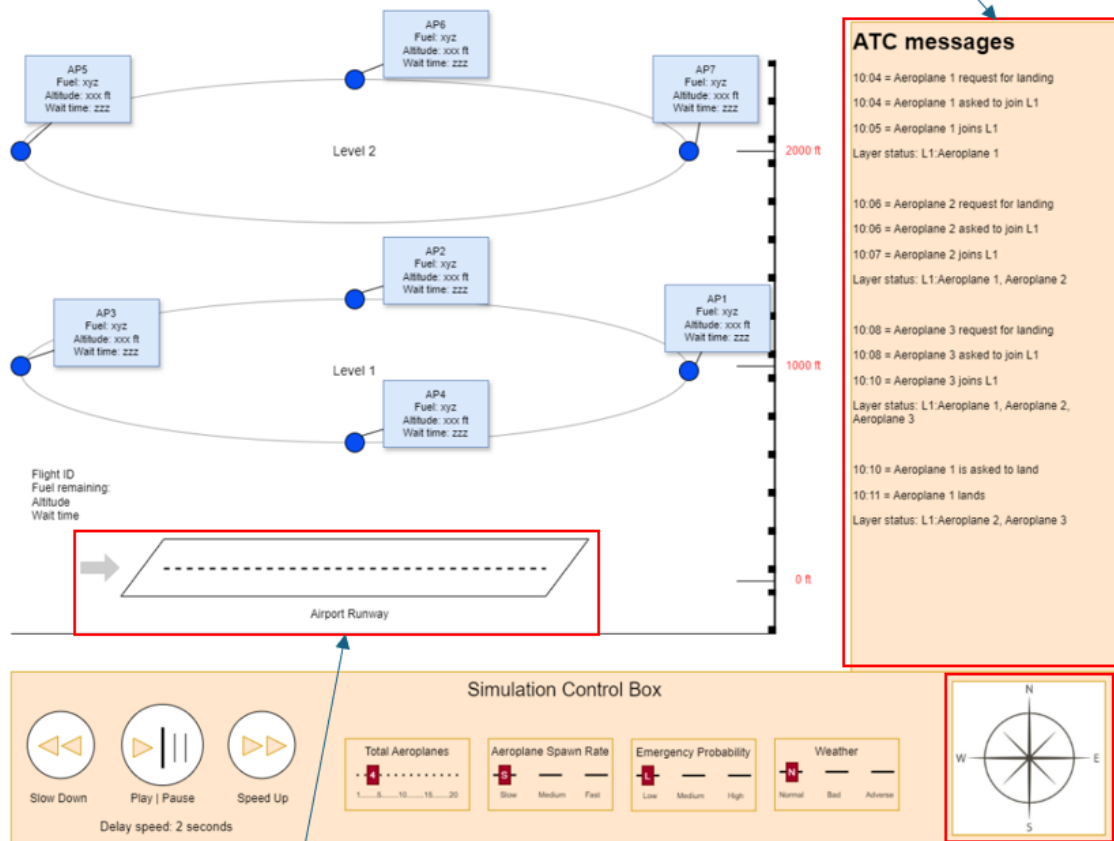
I had two different ideas for a user interface in mind, regarding the positions of the two holding patterns. Therefore, I created online sketches (in draw.io) for both ideas, shown below. The first sketch is of the start-up screen, then the following two sketches are of each idea of the 2D radar interface. Usability features are annotated.

2.3.1 Start-up screen



2.3.2 Radar interface (idea 1)

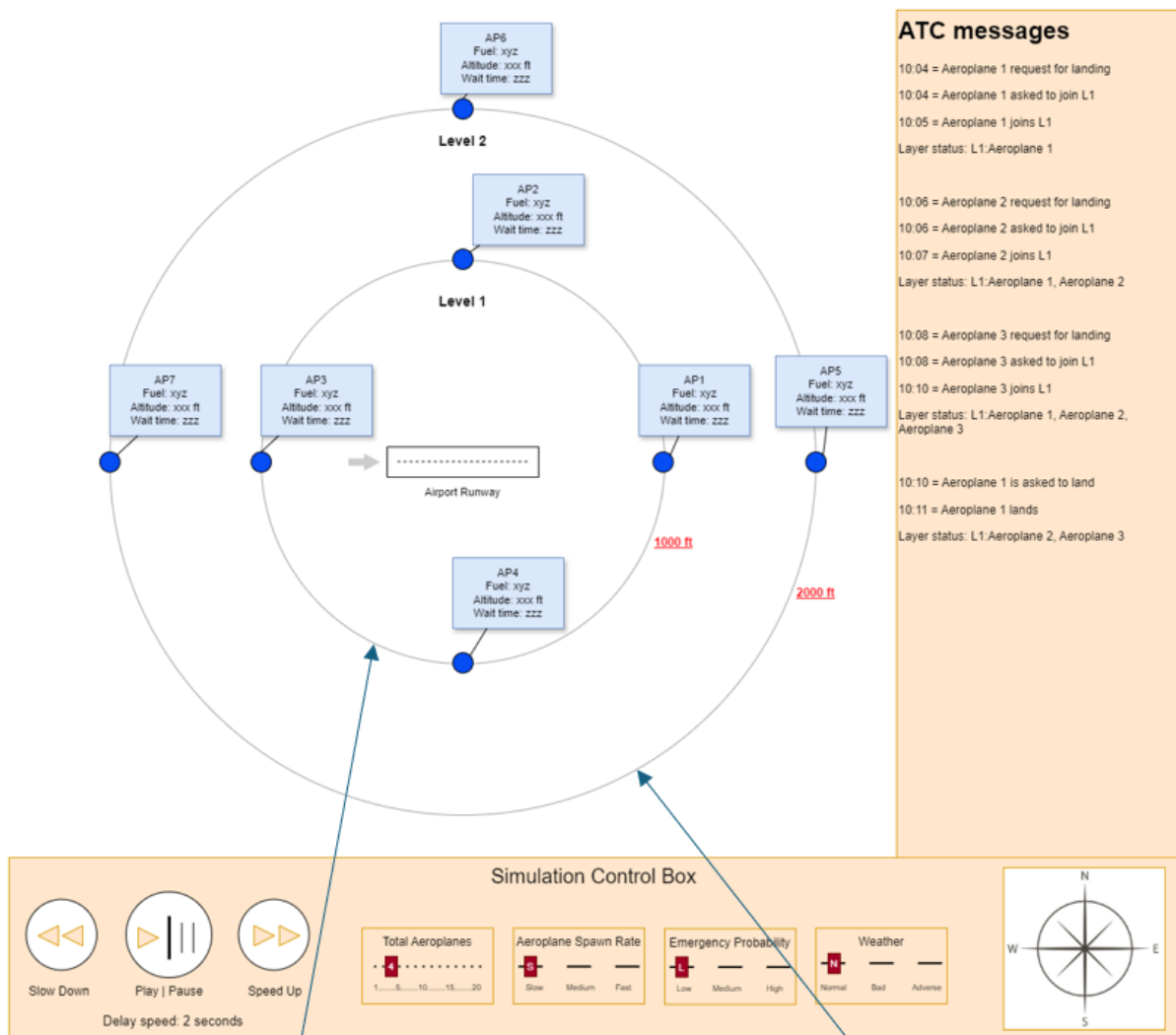
As per Requirement 5, there is an output box displaying messages to alert the user of new planes, the levels they are joining, landing permissions and landing status. This helps emulate the real-life messages a trainee will send when they become a controller



As per success criteria 2.2, a runway is displayed at the bottom of the radar as a visual aid so users can see where exactly planes need to land at. There is an arrow to its left pointing to the direction to approach (i.e. direction planes will land from)

As per Requirement 11, there is a compass in the bottom-right, in the same location as ATC-SIM's compass. This improves the user's sense of direction when determining aircraft headings, it is also a tool used by real controllers so improves the realism of the system

2.3.3 Radar interface (idea 2)



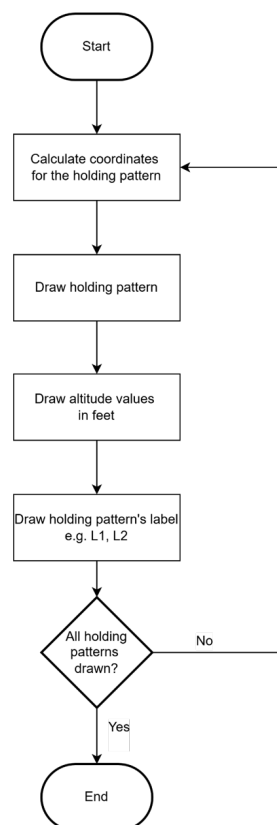
This concept still separates the circuits vertically by 1000 ft because regulations still apply, but this time the radar is a true birds-eye view so L2 is wider and surrounds L1. The advantage of this compared to Idea 1 is it is easier for the user to visualise as the screen is now a “true” radar, and L2 can fit more planes than L1, should traffic increase to the point where this is utilised. So, for a plane to drop from L2 to L1, it will descend in altitude but also move into the smaller ring before finally landing on the runway

I have now settled on option 2: to have a larger L2 ring surrounding L1 which surrounds the runway, with L2 at 2000 ft, L1 at 1000 ft, and the runway at 0 ft. This option is preferred as more planes can be accommodated on the larger L2 level. This improves the scalability of the solution to larger airfields that require a greater volume of aircraft in a holding pattern, such as London Heathrow (LHR).

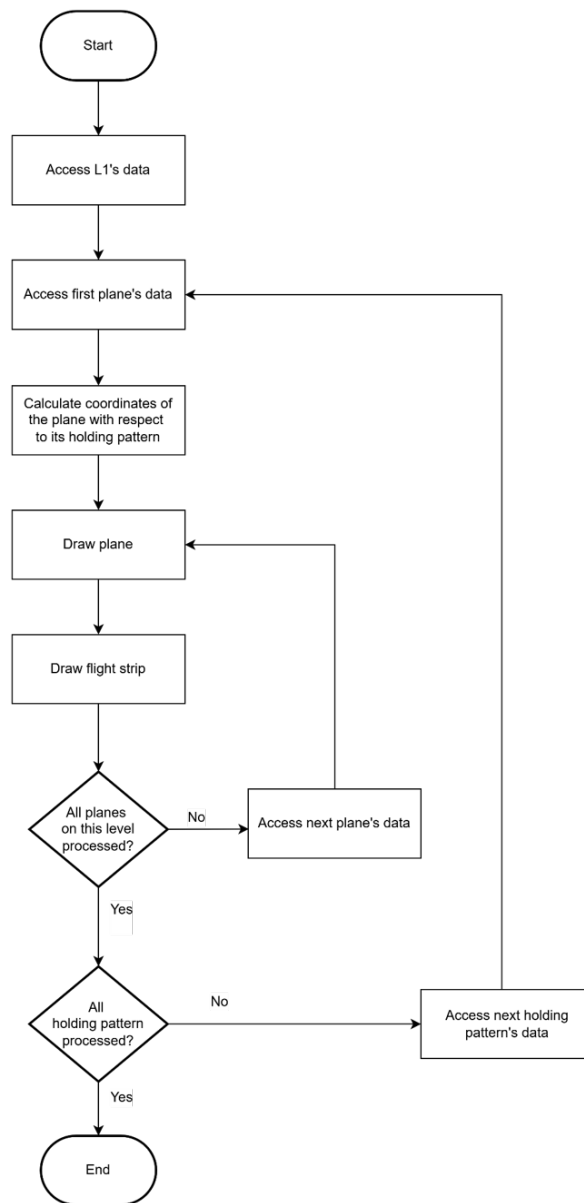
2.4 Algorithms

Flowcharts were produced for all 25 modules listed in the systems diagram. Some of the more complex flowcharts are shown below. Note that the arrow box indicates a subroutine (as draw.io didn't have an actual subroutine box).

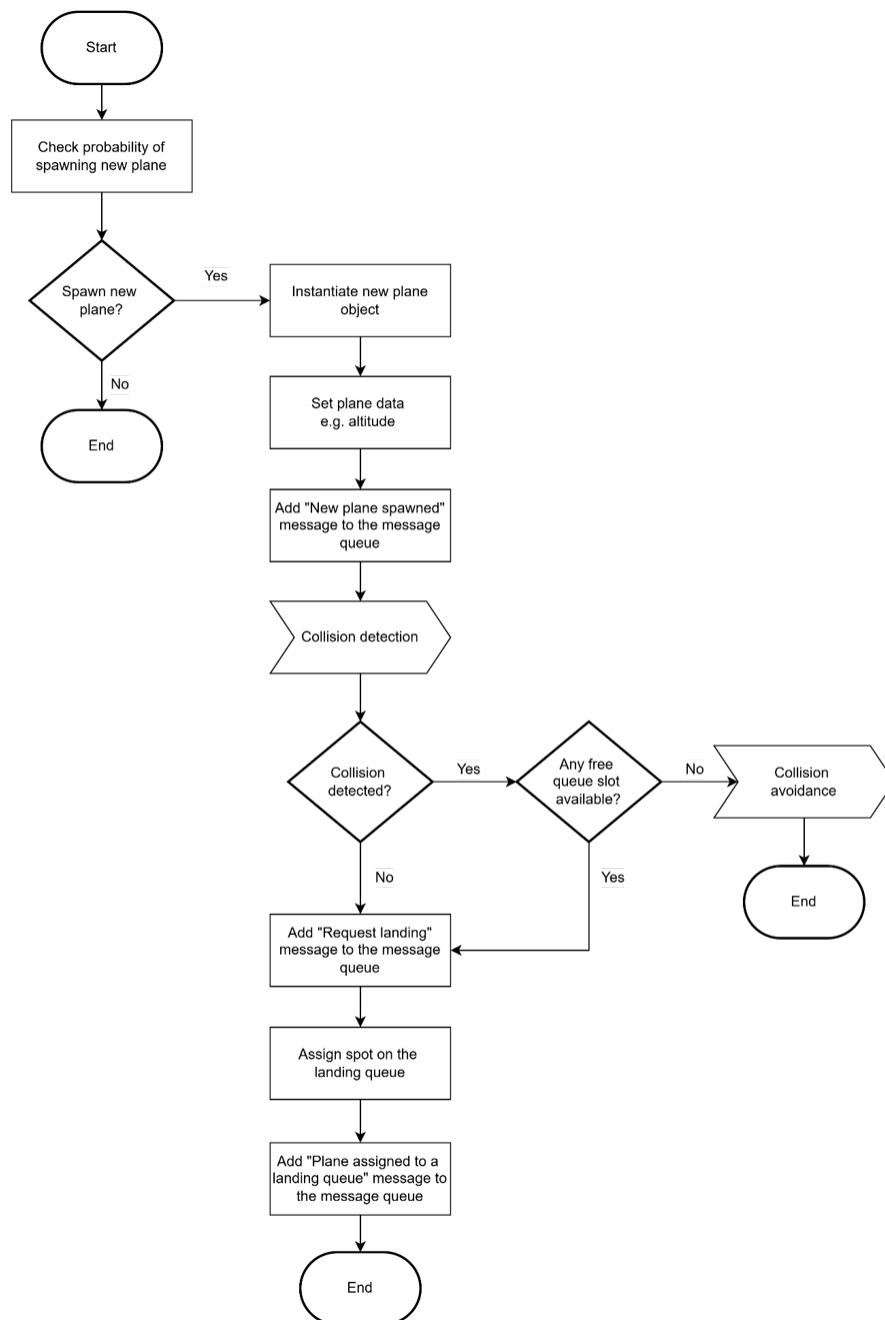
2.4.1 Render holding patterns



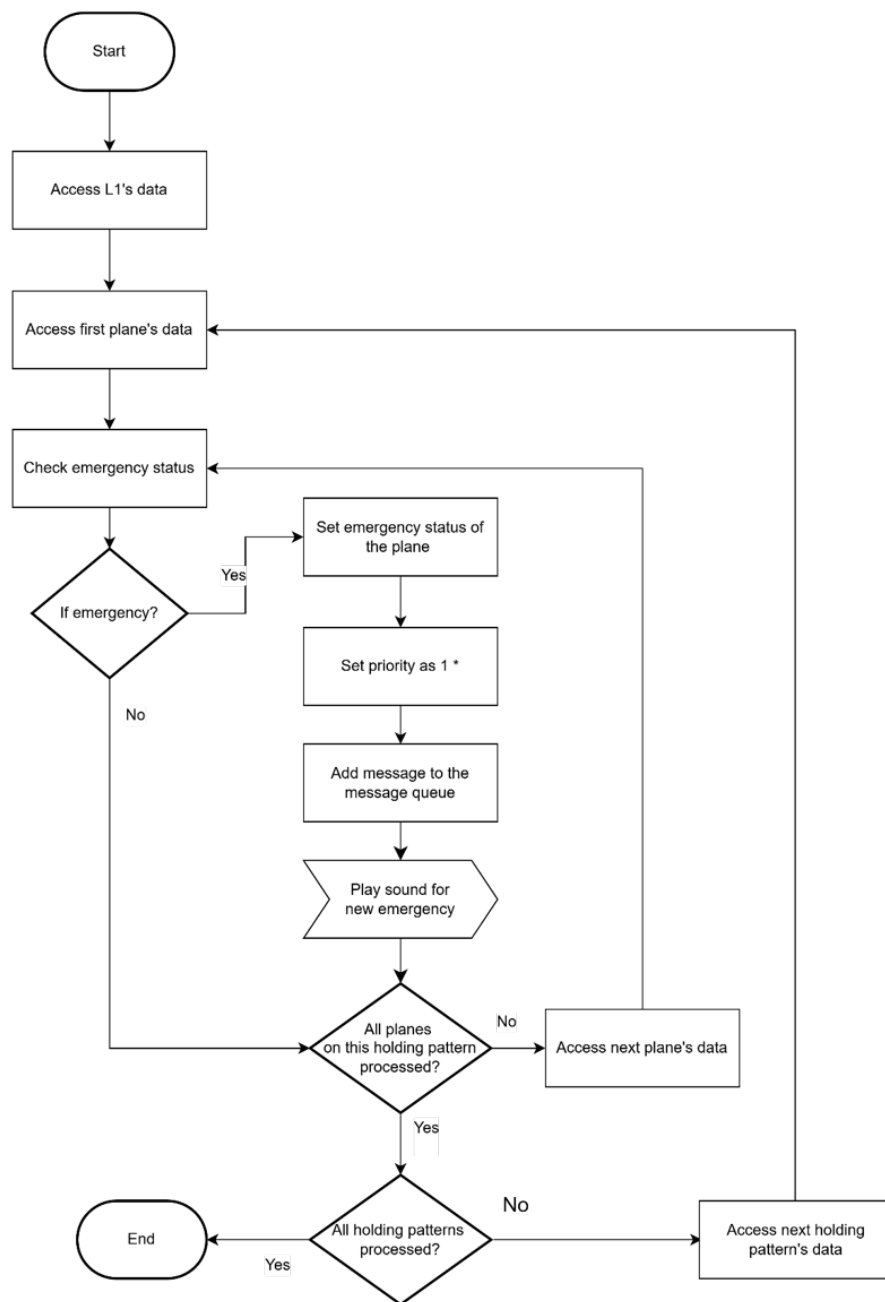
2.4.2 Render planes



2.4.3 Spawn new plane



2.4.4 Check for new emergency



* Priorities are defined as follows:

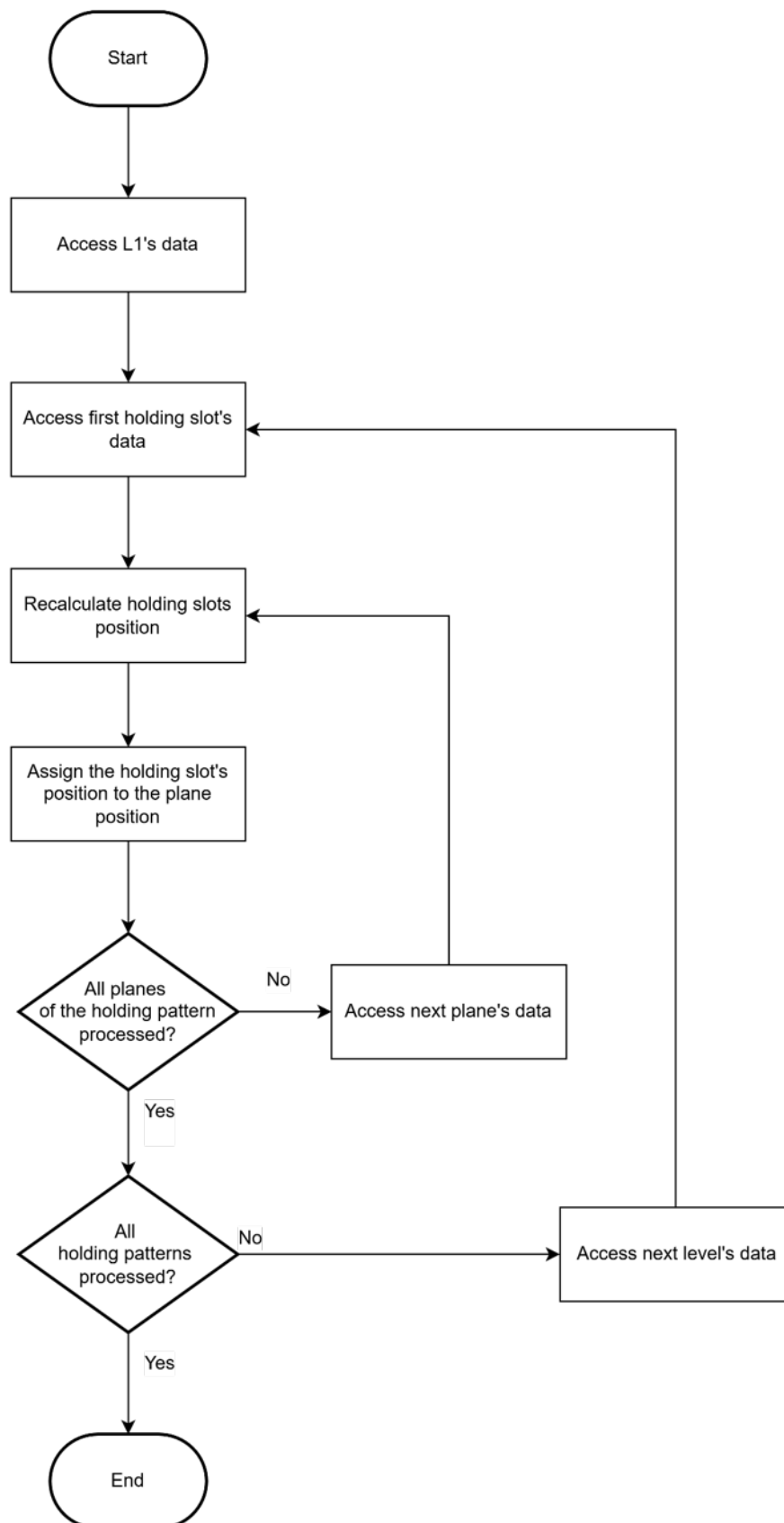
Priority 1: aircraft with an emergency

Priority 2: aircraft with no fuel but no emergency

Priority 3: aircraft with the longest waiting time to land but no emergency

Priority 4: aircraft with none of these conditions, first come first served

2.4.5 Recalculate plane coordinates



Chapter 3

Development

3.1 Overview

Development followed an **adaptive waterfall lifecycle** split into four iterations. Each iteration had a defined scope, followed by testing, debugging, and a review.

Iteration	Focus	Key deliverables
1	Start-up screen	StartupScreen class, five sliders, Start button, tkinter main window
2	Radar interface	Simulation class, main radar frame, message panel, control panel
3	Core simulation	Plane movement, fuel/wait tracking, landing removal, runtime buttons, messages
4	Advanced features	Weather, emergencies, priority queue, radar stats, compass

3.2 Iteration 1 — Start-up Screen

3.2.1 Scope

- StartupScreen class with all five setting sliders and a Start Simulation button.
- Establish the tkinter top-level window (`Tk()`).

3.2.2 Key Implementation Details

The `StartupScreen` class was created to encapsulate all visual elements of the start-up screen. The `__init__()` method calls sub-functions to build each widget in sequence:

Listing 3.1: `StartupScreen` `__init__` method structure

```
1 class StartupScreen:
2     def __init__(self, simulator):
```

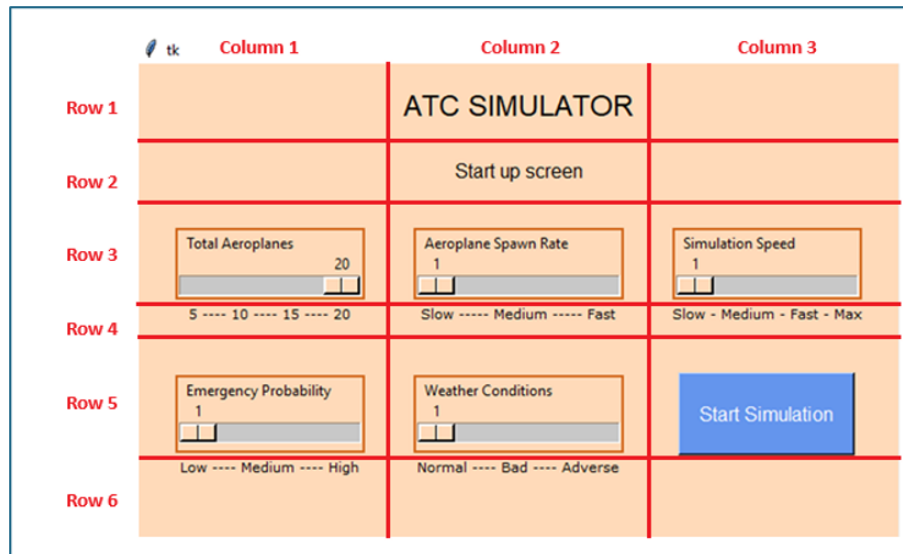
```

3         self.sim = simulator          # reference to Simulator object
4         self.create_main_frame()
5         self.create_title_labels()
6         self.create_total_planes_scaler()
7         self.create_plane_spawn_rate_scaler()
8         self.create_simulation_speed_scaler()
9         self.create_emergency_probability_scaler()
10        self.create_weather_scaler()
11        self.create_start_simulation_button()

```

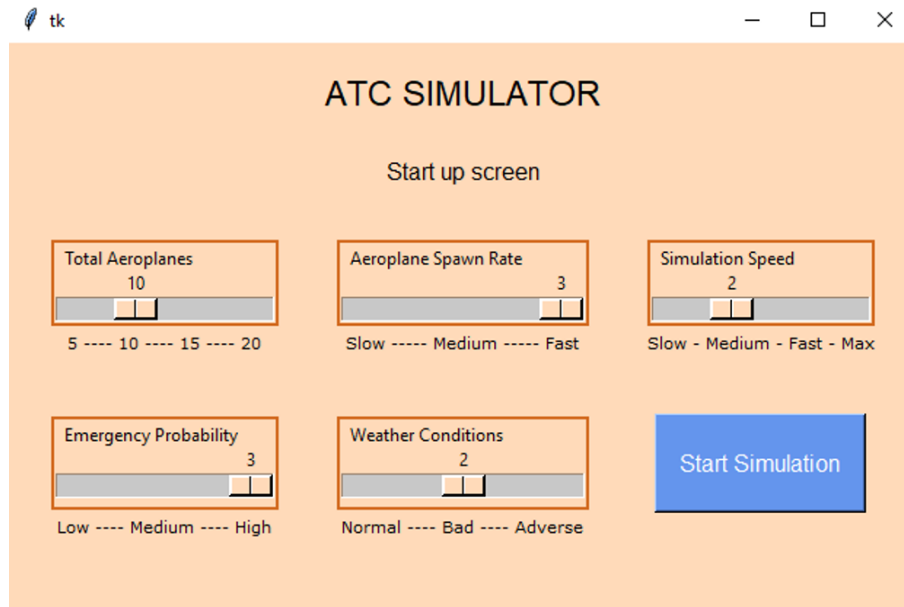
Each slider is a `tk.Scale` widget placed on a virtual grid. The slider currently being dragged highlights in red to give the user positive feedback. A legend below each slider shows the range of values (e.g. 5–20 for Total Aeroplanes).

Two options were considered for labelling the sliders: (1) help text descriptions in a separate section, and (2) image-based labels drawn directly above each slider. Option 2 was chosen for clarity and was implemented using image files stored in the `res/` directory (image below).



3.2.3 Iteration 1 — Review

The start-up screen rendered correctly with all five sliders and the Start button.



3.3 Iteration 2 — Radar Interface

3.3.1 Scope

- `Simulation` class with the main radar frame, message panel, and Simulation Control Box.
- Render the runway, holding patterns, output box, and on-screen controls.

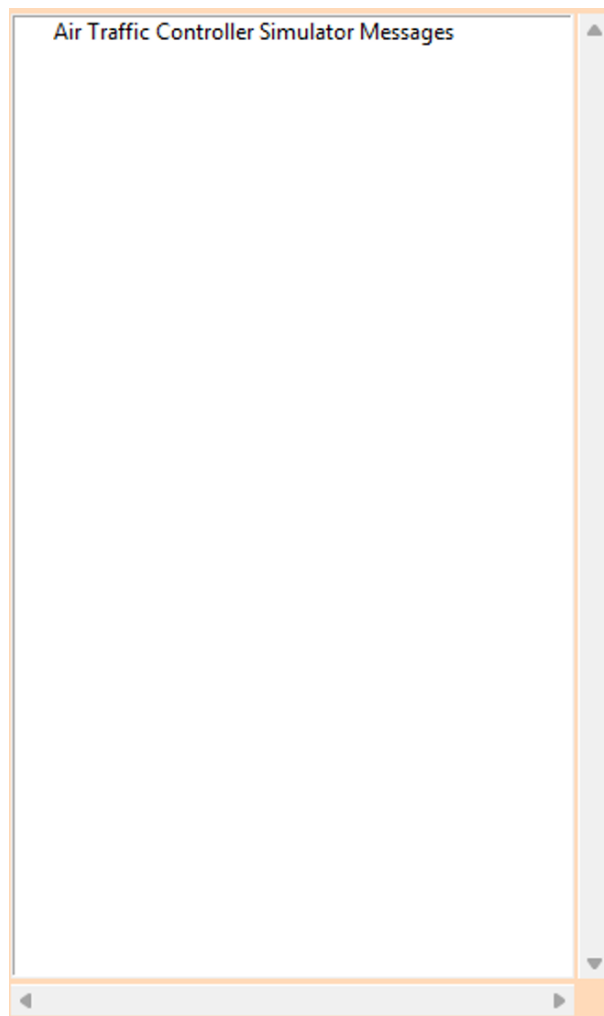
3.3.2 Key Implementation Details

The `Simulation` class uses a `tk.Canvas` for the radar panel, enabling dynamic drawing of all graphical elements each tick. The interface was partitioned into three regions using `tkinter`'s grid manager:

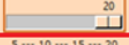
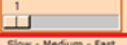
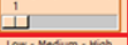
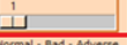
- **Radar panel (left, large)** — black background canvas for all dynamic content.



- **Message panel (right)** — white `tk.Listbox` labelled “Air Traffic Controller Simulator Messages”.



- **Simulation Control Box (bottom)** — runtime buttons (columns 1-3), runtime sliders (columns 4-7), and compass (column 8).

	Column 1	Column 2	Column 3	Column 4	Column 5	Column 6	Column 7	Column 8
Row 1	Simulation Control Box							
Row 2				Total Aeroplanes 	Aeroplane Spawn Rate 	Emergency Probability 	Weather Conditions 	
Row 3	Slow Down	Play Pause	Speed Up	5 --- 10 --- 15 --- 20	Slow - Medium - Fast	Low - Medium - High	Normal - Bad - Adverse	

3.3.3 Debugging & Error Handling

Error 1 Exception warning when accessing image files. Resolved by adjusting the image file path to be relative to the script's directory using `os.path`.

Error 2 Start Simulation button did not transition to the radar screen. Resolved by correctly binding the button command to the `start_simulation()` method of the `Simulator` class.

3.3.4 Iteration 2 — Review

Runway, holding patterns, output box, and on-screen controls all rendered correctly. Three planes appeared as lime-green dots on first load, confirming the simulation screen was functional. A stakeholder suggested changing the background colour of the radar panel from grey to black to better reflect real ATC colour schemes. This change is shown below:



3.4 Iteration 3 — Core Simulation Logic

3.4.1 Scope

- Plane movement along holding-pattern orbits using trigonometric functions.
- Fuel and wait-time tracking per plane.
- Plane landing and removal from the radar.
- Pause/Play, Speed Up, Slow Down runtime buttons.
- Spawn, land, and end-of-simulation output-box messages.

3.4.2 Plane Class

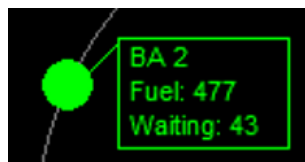
The `Plane` class encapsulates all per-aircraft state and behaviour:

Listing 3.2: Plane class core methods

```

1 class Plane:
2     def __init__(self, flight_id, level, slot):
3         self.flight_id = flight_id
4         self.level      = level          # 1 (lower) or 2 (upper)
5         self.slot       = slot          # HoldingSlot object
6         self.fuel       = random.randint(40, 100)
7         self.wait_time  = 0
8         self.emergency  = False
9         self.angle      = slot.start_angle
10
11     def set_angles(self): ...           # compute angular step per tick
12     def calculate_position(self): ...   # x, y from angle & radius
13     def draw_plane_graphics(self): ...  # dot + flight strip on canvas
14     def move_plane_graphics(self): ...  # update canvas item positions
15     def land_plane(self): ...          # animate descent to runway

```



3.4.3 Message Class

The `Message` class stores all output-box message templates:

Listing 3.3: Message class methods

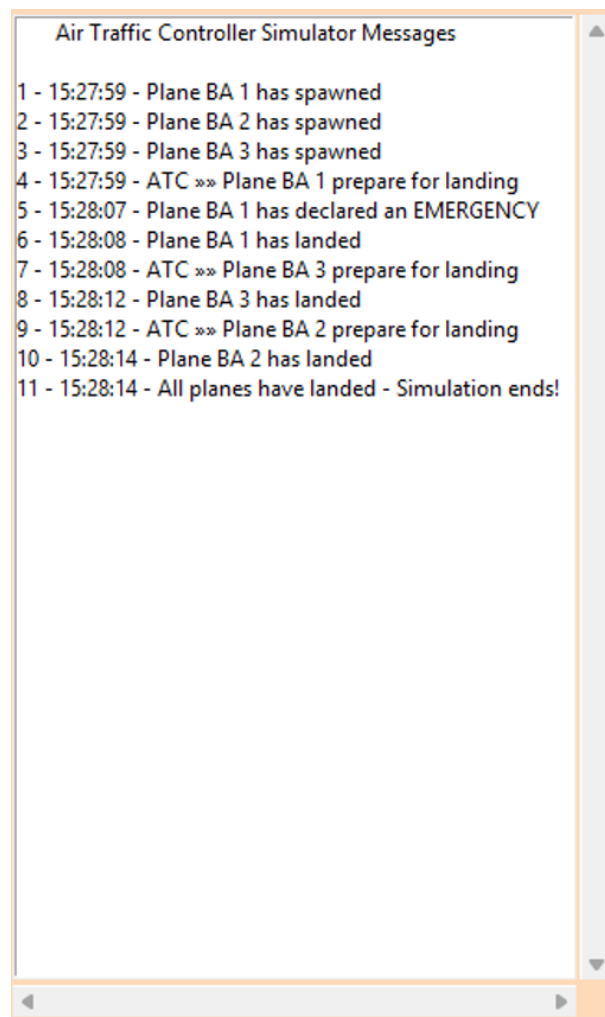
```
1 class Message:
2     def create_plane_spawned_message(self, fid): ...
3     def create_prepare_for_landing_message(self, fid): ...
4     def create_plane_landed_message(self, fid): ...
5     def create_level_changed_message(self, fid, level): ...
6     def create_plane_emergency_message(self, fid): ...
7     def create_end_of_simulation_message(self): ...
```

The structure of a message string is as follows:

[Message ID] - [timestamp] - [Plane ID] [the message string]

There are a few exceptions to the above rule. These exceptions are related to the scenarios when a general message needs to be broadcasted:

- 4 – 15:27:59 – ATC » Plane BA1 prepare for landing
- 11 – 15:27:59 – All planes have landed – Simulation ends!



3.4.4 Debugging & Error Handling

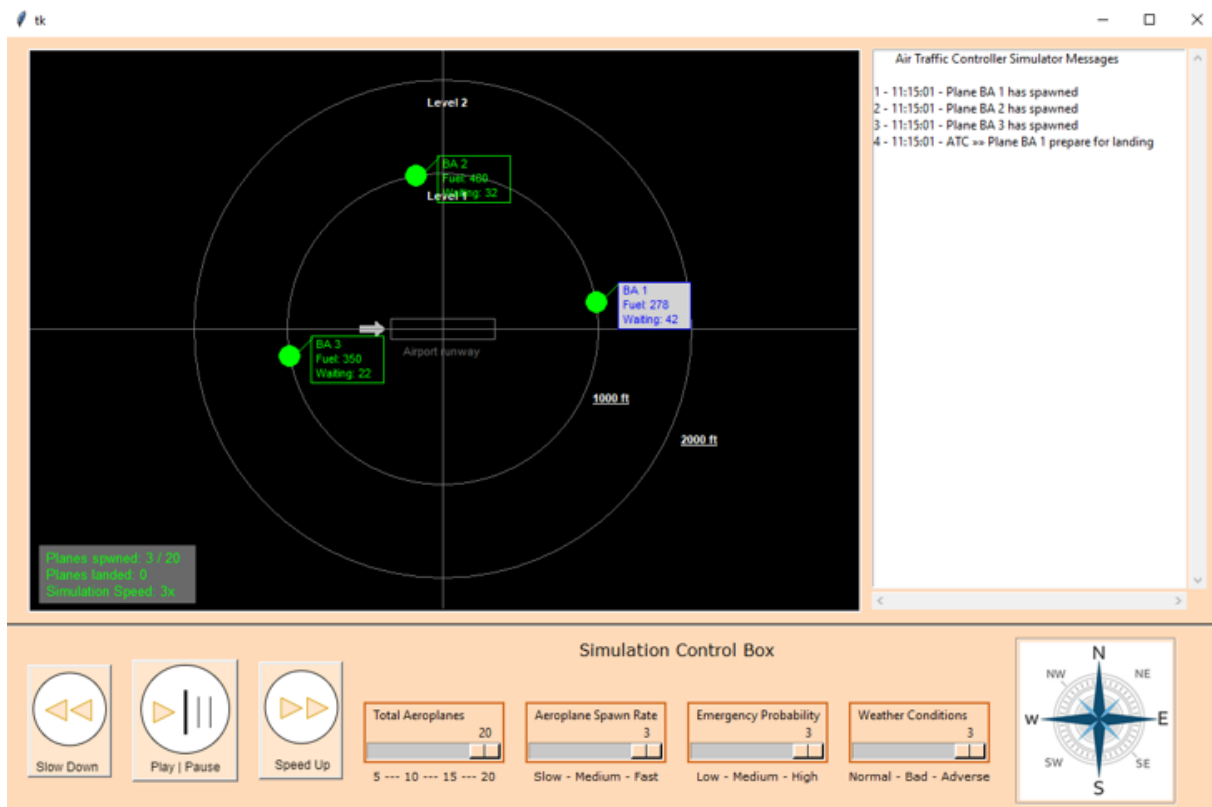
Error 1 Landed plane not removed once on runway. The `process_landed_plane()` check was triggered too early. Fixed by tightening the runway-arrival coordinate threshold.

Error 2 Pause/Play button did not unpause. The boolean flag `paused` was being set but not checked in the run-engine loop. Fixed by adding a conditional branch at the start of `run_engine()`.

Error 3 Simulation did not stop when all planes had landed. The termination condition compared the wrong counter variable. Fixed by referencing the correct `total_planes_landed` counter.

3.4.5 Iteration 3 — Review

Plane movement, flight strips, runtime buttons, and all message types functioned correctly. All Iteration 3 test cases passed.



3.5 Iteration 4 — Advanced Features

3.5.1 Scope

- Weather conditions affecting the simulation visually.
- Emergency generation and detection.
- Priority landing-queue algorithm.
- Real-time radar stats box (total planes, landed, in circuit).
- Compass widget in the Simulation Control Box.

3.5.2 Priority Queue Algorithm

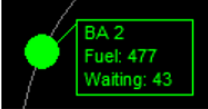
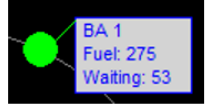
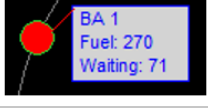
The `sort_criteria()` function ranks all planes in the circuit using a composite score. Planes are sorted in the following order of precedence:

1. Emergency status (emergency = highest priority).
2. Fuel remaining (lower fuel = higher priority).
3. Holding level (Level 1 = higher priority than Level 2).
4. Wait time (longer wait = higher priority).

The top-ranked plane is coloured blue on the radar to indicate it has been issued landing clearance.

Listing 3.4: `sort_criteria()` composite key

```
1 def sort_criteria(self, plane):
2     return (
3         0 if plane.emergency else 1,    # emergency first
4         plane.fuel,                     # lowest fuel first
5         plane.level,                    # L1 before L2
6         plane.wait_time                  # longest wait first
7     )
```

Visual representation	Description
	- The plane has no emergency - The plane is awaiting landing instruction
	- The plane has no emergency - The plane has been given instruction to land. There can be only one plane at any given time to receive landing instructions.
	- The plane has an emergency - The plane is awaiting landing instruction
	- The plane has an emergency - The plane has been given instruction to land. There can be only one plane at any given time to receive landing instructions.

3.5.3 Debugging & Error Handling

Error 1 Plane did not declare emergency when low on fuel. Emergency threshold was not being evaluated in `check_and_generate_emergency()`. Fixed by adding a fuel-level branch to the emergency check.

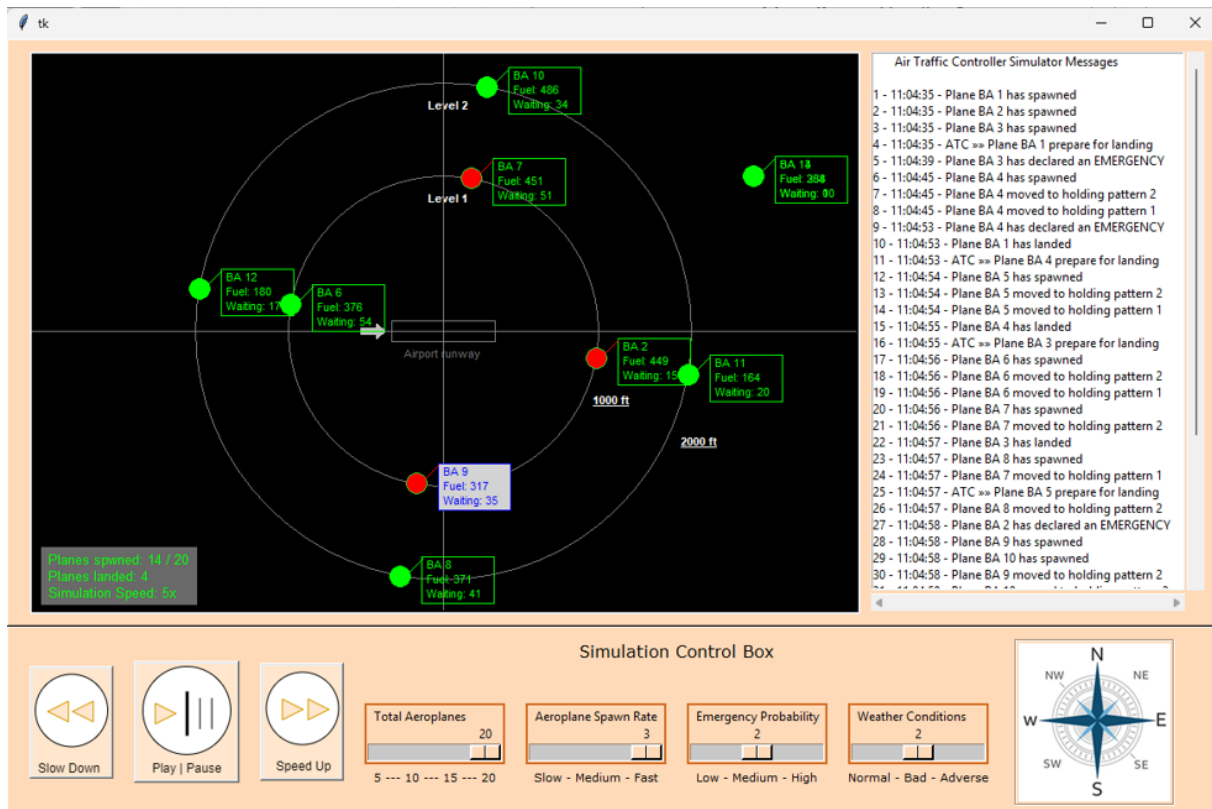
Error 2 Changing the total-plane slider did not update the radar info box. The info-box update call was missing from the slider's command callback. Fixed by adding `update_radar_screen_stats()` to the callback.

Error 3 Incorrect plane landing priority when multiple criteria tied. Composite sort key was evaluated in wrong order. Fixed by reordering the tuple elements in `sort_criteria()`.

Error 4 Newly spawned planes rendered on top of each other. All planes were being spawned at the same fixed coordinate. Fixed by assigning each plane to an empty `HoldingSlot` with a distinct start angle.

3.5.4 Iteration 4 — Review

Emergency detection, priority queuing, weather conditions, and radar stats all functioned correctly. Message window regularly updates with key events e.g. plane landing, moving to lower levels, emergencies etc. Collision detection and avoidance were not implemented due to time constraints.



Chapter 4

Testing

4.1 Acceptance Testing (Post-Development)

Acceptance testing was conducted with two stakeholders using a recorded video walk-through. The table below summarises the result (extent of implementation) against each requirement with notes. Each requirement will be discussed in further detail in Chapter 5 (Evaluation).

No.	Requirement	Result	Notes
1	Start-up screen	Yes	All three sub-criteria met
2	2D radar GUI	Yes	Runway and aircraft dots render correctly
3	Holding patterns	Yes	Two concentric circles, 1000 ft separation labelled
4	Settings sliders	Yes	All three runtime sliders function
5	Communications box	Partial	Collision warning message not implemented
6	Program end	Yes	Halts when all planes landed
7	Flight strips	Yes	FlightID, fuel, wait time displayed
8	Sound effects	Partial	Emergency sound works; collision sound not implemented
9	Runtime buttons	Yes	Pause, Speed Up, Slow Down all functional
10	Collision detection	No	Not implemented due to time constraints
11	Heading indicator	Partial	Compass renders; wind arrow not implemented
12	Plane removal	Yes	Planes despawned on landing

Chapter 5

Evaluation

5.1 Usability Features

5.1.1 Start-up Screen

The screen is simple and easy to navigate, featuring five labelled sliders with legends and a clearly positioned Start Simulation button. One stakeholder confirmed it closely matched the original design drawing. However, another noted a minor inconsistency between the numeric value shown above a dragged slider and the text legend below it but considered it inconsequential.

5.1.2 Main Radar Screen

The radar occupies the majority of the screen. A runway at centre with a directional arrow, two labelled holding patterns, a white output box on the right, and a Simulation Control Box at the bottom collectively provide all the information an ATC instructor and student needs. All stakeholders agreed the layout was intuitive. Criticism included: the large crosshair obscuring the runway and the compass being redundant without a wind arrow.

5.1.3 Aircraft Recognition

Each plane is represented by a colour-coded dot with an adjacent flight strip detailing information about that flight, allowing users to distinguish between aircraft (mainly by flightID). The various dot and flight strip colours are explained in Chapter 3 (Development) section 3.5 (Iteration 4). All stakeholders praised this approach for enabling quick visual identification of critical aircraft among high traffic volumes.

5.2 How Well Does the Solution Match the Success Criteria?

5.2.1 Requirements Fully Met (8/12)

Requirements 1 (start-up screen), 2 (radar interface), 3 (2x holding patterns), 4 (settings sliders), 6 (program end), 7 (flight strips), 9 (runtime buttons), and 12 (plane removal) were fully met. All Must-Have requirements (1–6) were met, except Requirement 5 (communications box) which was partially met.

5.2.2 Requirements Partially Met (3/12)

Requirement 5 — Communications box All implemented message types (spawn, land, emergency, level change, end) are present. The collision-warning message could not be added as collision detection (Req. 10) was not implemented. If given more time: implement Req. 10 first, then add the collision message type — this would be straightforward as the message infrastructure already exists.

Requirement 8 — Sound effects Emergency alarm functions correctly. Collision sound not implemented for the same reason as above. If given more time: add a distinct alarm sound triggered alongside the collision detection output message.

Requirement 11 — Heading indicator Compass renders with cardinal directions. Wind direction arrow not implemented. If given more time: add a function in the run engine that checks the weather variable each tick and renders a blue directional arrow on the compass when weather is Bad or Adverse.

5.2.3 Requirements Not Met (1/12)

Requirement 10 — Collision detection & avoidance Not implemented due to time constraints in Iteration 4. This reduces some realism, though the absence of collision handling does not cause actual collisions in the current system as planes are always assigned to distinct holding slots. If given more time: (1) spawn planes outside the holding patterns so they must queue independently, increasing collision risk; (2) implement `sort_criteria()` to detect converging paths; (3) print collision warning to output box; (4) redirect the left-hand aircraft using aviation’s “right of way” convention; (5) animate the aircraft moving into a temporary holding position.

5.3 Maintenance

Good coding practice was observed throughout:

- All subroutines and calculations are commented with their purpose.

- Consistent indentation (auto-assisted by VS Code).
- Descriptive variable and subroutine names (e.g. `emergency_probability`, `spawn_new_plane`).

Future maintenance features (for professional distribution):

- In-app bug-reporting form feeding into a patch release pipeline.
- Package as a standalone application (removing the need for Python and VS Code).

5.4 Limitations and Future Development

No.	Limitation	How it would be addressed
1	3D radar interface	Research a library and implement 3D rendering of surrounding terrain, including high-relief features such as mountain ranges affecting approach paths.
2	Approach aids (ILS)	Add output messages displaying horizontal and vertical offset from optimal descent path; draw ILS path (converging lines) on the radar (Fig. 5.1).
3	Multiple runways	Add a start-up option for runway count and configuration (parallel as at LHR, crossing as at SFO); dynamically generate holding patterns per runway.
4	Aircraft teleportation	Animate plane dots along an elliptical transition arc between holding levels rather than teleporting.

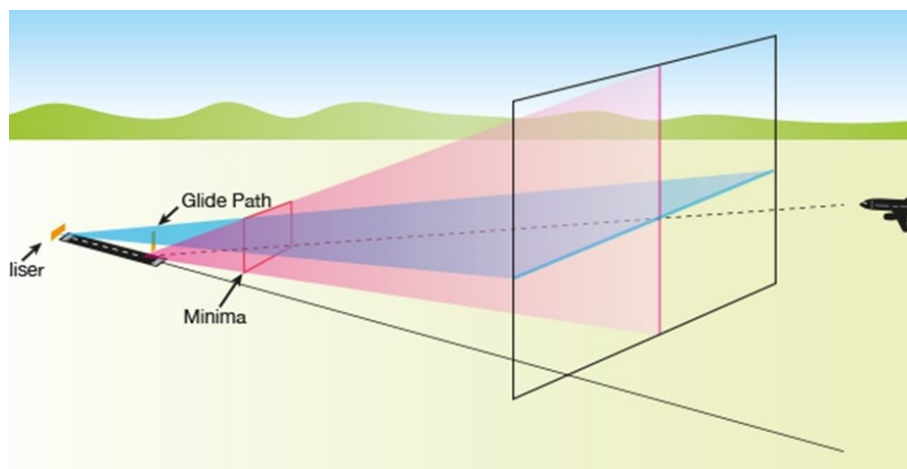


Figure 5.1: Visualisation of how an ILS works. Pink wave = vertical guidance, blue waves = horizontal guidance to pilots